Assignment No. 10

Name: Shubham Shivaji Phad
Class & Division: LY AIA 03 Batch B
Roll-No:48
Enrollment Number: ADT23SOCB1106

Title

Develop a text classification system using Machine Learning algorithms such as Naïve Bayes, SVM, or Logistic Regression to automatically categorize documents into predefined classes.

Aim

To develop a text classification system using machine learning algorithms to automatically categorize documents into predefined classes.

Objectives

- Understand text classification using supervised machine learning.
- Convert text documents into numerical TF-IDF features.
- Implement Naïve Bayes, Logistic Regression, and SVM classifiers.
- Evaluate classification performance using accuracy and classification reports.
- Predict the class of a new document.

Theory

Text classification assigns a document to a predefined category based on its textual content. The text is first converted into numerical features using TF-IDF. A classifier then learns the relationship between these features and the target classes.

Naïve Bayes uses class and word probabilities. **Logistic Regression** learns class probabilities using a linear decision boundary. **Support Vector Machine (SVM)** finds a decision boundary that separates classes with a maximum margin. These methods are commonly used for document and text classification.

Software/Libraries Required

- Python 3.x

- Google Colab / Jupyter Notebook
- NumPy
- Pandas
- scikit-learn

Installation Commands

```
!pip install numpy pandas scikit-learn -q
```

Algorithm/Steps

1. Prepare labeled documents belonging to predefined classes.
2. Split the documents into training and testing sets.
3. Convert documents into TF-IDF feature vectors.
4. Train Naïve Bayes, Logistic Regression, and SVM classifiers.
5. Predict the classes of test documents.
6. Calculate accuracy and classification reports.
7. Use the trained models to classify a new document.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report

documents = [
    'The team won the cricket match with excellent batting',
    'The football players scored three goals in the match',
    'The coach prepared the team for the championship',
    'The government announced a new education policy',
    'Parliament discussed the new national budget',
    'The election results were announced by the government',
    'The company released a new smartphone with advanced features',
    'Artificial intelligence is transforming technology companies',
    'The software update improves computer security',
    'The player scored a goal in the final match',
    'The minister presented a new policy in parliament',
    'The technology company developed a machine learning system'
]

labels = [
    'Sports', 'Sports', 'Sports',
    'Politics', 'Politics', 'Politics',
    'Technology', 'Technology', 'Technology',
    'Sports', 'Politics', 'Technology'
]

df = pd.DataFrame({'Document': documents, 'Class': labels})
df
```

	Document	Class
0	The team won the cricket match with excellent ...	Sports
1	The football players scored three goals in the...	Sports
2	The coach prepared the team for the championship	Sports
3	The government announced a new education policy	Politics
4	Parliament discussed the new national budget	Politics
5	The election results were announced by the gov...	Politics
6	The company released a new smartphone with adv...	Technology
7	Artificial intelligence is transforming techno...	Technology
8	The software update improves computer security	Technology
9	The player scored a goal in the final match	Sports
10	The minister presented a new policy in parliament	Politics
11	The technology company developed a machine lea...	Technology

```
X_train, X_test, y_train, y_test = train_test_split(
    df['Document'], df['Class'], test_size=0.25, random_state=42,
         stratify=df['Class']
)

vectorizer = TfidfVectorizer(stop_words='english')
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

models = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'SVM': LinearSVC()
}

results = {}
for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    predictions = model.predict(X_test_tfidf)
    results[name] = accuracy_score(y_test, predictions)
    print(f'\n{name}')
    print('Accuracy:', round(results[name], 4))
    print(classification_report(y_test, predictions, zero_division=0))
```

```
Naive Bayes
Accuracy: 0.6667
              precision    recall  f1-score   support

    Politics       0.50      1.00      0.67         1
      Sports       1.00      1.00      1.00         1
  Technology       0.00      0.00      0.00         1

    accuracy                           0.67         3
   macro avg       0.50      0.67      0.56         3
```

```
weighted avg       0.50      0.67      0.56         3


Logistic Regression
Accuracy: 1.0
              precision    recall  f1-score   support

    Politics       1.00      1.00      1.00         1
      Sports       1.00      1.00      1.00         1
  Technology       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


SVM
Accuracy: 1.0
              precision    recall  f1-score   support

    Politics       1.00      1.00      1.00         1
      Sports       1.00      1.00      1.00         1
  Technology       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

```python
# Predict the class of a new document using the best accuracy model
best_name = max(results, key=results.get)
best_model = models[best_name]

new_document = ['The football team won the championship match']
new_features = vectorizer.transform(new_document)
prediction = best_model.predict(new_features)[0]

print('Best Model:', best_name)
print('New Document:', new_document[0])
print('Predicted Class:', prediction)
```

```
Best Model: Logistic Regression
New Document: The football team won the championship match
Predicted Class: Sports
```

Program with Output

The program displays the TF-IDF based classification results for Naïve Bayes, Logistic Regression, and SVM. It also displays accuracy, classification reports, and the predicted class for a new document.

Typical output:

- Classifier accuracy values for the test set.
- Precision, recall, and F1-score for the predefined classes.
- The new document is classified as **Sports** because it contains strong sports-related terms.

Exact accuracy values may vary with the train-test split and dataset.

Conclusion

The text classification system was successfully implemented using TF-IDF features with Naïve Bayes, Logistic Regression, and SVM classifiers. The models were evaluated using accuracy and classification reports and were also used to categorize a new document into a predefined class. This demonstrates the application of supervised machine learning for automatic document classification.